

GCP Compute Engine — Full Deployment Guide

Step-by-step instructions to deploy the Conuar Inspection Webapp (Django + MySQL) on a Google Cloud Platform VM.

 Generated 2026-06-12  Google Cloud Platform  Python 3 + Django 4.2  MySQL 8.0

TABLE OF CONTENTS

01	Prerequisites	02	Create the VM
03	Open firewall port 8000	04	SSH into the VM
05	Install system packages	06	Install Docker (optional)
07	Configure MySQL	08	Clone the repository
09	Virtualenv & dependencies	10	Create the .env file
11	Migrate & collect static	12	Create required directories
13	Django as a systemd service	14	ETL monitor service
15	Open in browser	16	Daily commands
17	Summary		



Prerequisites — What you need before starting

Local setup checklist

- A GCP account with billing enabled
- The GitHub repository URL for the Conuar webapp
- The `gcCloud` CLI installed locally — or use **Cloud Shell** in the browser (zero install needed)

Tip: All commands from Step 1 onward can be run directly from **GCP Cloud Shell** (the terminal icon in the GCP Console top bar) without installing anything locally.

01

Create the VM

e2-medium — 1 vCPU, 4 GB RAM, Debian 12

Run this in Cloud Shell or in a local terminal with `gcloud` configured:

Create Compute Engine instance

Copy

```
gcloud compute instances create conuar-webapp \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --boot-disk-size=30GB \
  --zone=us-central1-a \
  --tags=http-server
```

Machine type: e2-medium = 1 vCPU + 4 GB RAM. Sufficient for Django + MySQL on low/medium traffic. Upgrade to e2-standard-2 (2 vCPU + 8 GB) if you add TensorFlow inference or heavier ETL workloads.

02

Open firewall port 8000

Allow inbound traffic to Django's default port

Create firewall rule

Copy

```
gcloud compute firewall-rules create allow-django \
  --allow=tcp:8000 \
  --target-tags=http-server \
  --description="Django runserver port"
```

03

SSH into the VM

All following commands run inside the VM

Connect via gcloud SSH

Copy

```
gcloud compute ssh conuar-webapp --zone=us-central1-a
```

All steps below run inside the VM terminal.

04

Install system packages

Python, MySQL, Git, and build tools

apt-get install

Copy

```
sudo apt-get update && sudo apt-get upgrade -y

# Python runtime and venv support
sudo apt-get install -y python3 python3-pip python3-venv python3-dev

# MySQL server + C client headers (needed by pymysql/mysqlclient)
sudo apt-get install -y mysql-server default-libmysqlclient-dev pkg-config

# Git
sudo apt-get install -y git

# C build tools (needed by lxml, Pillow, cffi, cryptography)
sudo apt-get install -y gcc build-essential libssl-dev libffi-dev \
    libjpeg-dev zlib1g-dev libpq-dev
```

05

Install Docker (optional)

Skip if you're deploying directly without containers

Install Docker Engine

Copy

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker          # apply group without logout
docker --version       # verify: Docker version 27.x.x
```



Configure MySQL

Start the server, secure it, create database and user

START AND ENABLE MYSQL

systemctl

Copy

```
sudo systemctl start mysql
sudo systemctl enable mysql

# Secure installation: set root password, remove anonymous users
sudo mysql_secure_installation
```

When prompted: set root password → **yes**, remove anonymous users → **yes**, disallow root remote login → **yes**, remove test DB → **yes**.

CREATE DATABASE AND APPLICATION USER

Connect to MySQL as root

Copy

```
sudo mysql -u root -p
```

Inside the MySQL shell

Copy

```
CREATE DATABASE conuar_webapp
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'conuar'@'localhost'
  IDENTIFIED BY 'YourStrongPassword123!';

GRANT ALL PRIVILEGES ON conuar_webapp.* TO 'conuar'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

Replace `YourStrongPassword123!` with a real password. You will need it again in the `.env` file in Step 9.

07

Clone the repository

Deploy into /opt/conuar

```
git clone
```

[Copy](#)

```
cd /opt
sudo mkdir conuar && sudo chown $USER:$USER conuar
cd conuar

# Public repo:
git clone https://github.com/YOUR_USER/YOUR_REPO.git .

# Private repo (use a GitHub personal access token):
# git clone https://<TOKEN>@github.com/YOUR_USER/YOUR_REPO.git .
```

Tip: Generate a GitHub fine-grained token at *Settings* → *Developer settings* → *Personal access tokens* with read-only access to the repository if it is private.

08

Create the virtualenv and install dependencies

Isolated Python environment with pinned packages

```
venv + pip install
```

[Copy](#)

```
cd /opt/conuar/Conuar/conuar_webapp

python3 -m venv venv
source venv/bin/activate

pip install --upgrade pip
pip install -r requirements.txt
```

Note: If `requirements.txt` includes `tensorflow` or `opencv-python`, installation may take 5–10 minutes and ~2 GB of disk. On a 30 GB boot disk this is fine.

09

Create the `.env` file

Environment variables for Django — never commit this file

Open nano editor

Copy

```
nano /opt/conuar/Conuar/conuar_webapp/.env
```

Contents of `.env`

Copy

```
# Django
SECRET_KEY=your-long-random-secret-key-here
DEBUG=False
ALLOWED_HOSTS=YOUR_VM_EXTERNAL_IP,localhost

# Database
DB_NAME=conuar_webapp
DB_USER=conuar
DB_PASSWORD=YourStrongPassword123!
DB_HOST=localhost
DB_PORT=3306

# Email
EMAIL_HOST_USER=arbyteconuar@gmail.com
EMAIL_HOST_PASSWORD=your-gmail-app-password
INSPECTION_PROCESSED_EMAIL_RECIPIENTS=carlosd2.rodriguez@gmail.com
INSPECTION_PROCESSED_EMAIL_ENABLED=true
```

Find `YOUR_VM_EXTERNAL_IP` in **GCP Console** → **Compute Engine** → **VM instances** → **External IP** column.

Generate a secure `SECRET_KEY`: Run `python3 -c "import secrets; print(secrets.token_urlsafe(50))"` inside the VM.

10

Run migrations and collect static files

Initialize the database schema and gather static assets

Django management commands

Copy

```
cd /opt/conuar/Conuar/conuar_webapp
source venv/bin/activate

python manage.py migrate
python manage.py collectstatic --noinput

# Create the Django admin superuser
python manage.py createsuperuser
```

11

Create required directories

Logs, ETL inputs, photo staging, PDFs

mkdir

Copy

```
mkdir -p logs
mkdir -p etl/NodeRed/plc_reads
mkdir -p STAGING PROCESSED pdfs
```

12

Set up Django as a systemd service

Auto-start on boot + auto-restart on crash

Systemd is strongly preferred over cron for long-running processes: it restarts on crash, logs via `journalctl`, and starts before any user logs in.

Create service file

Copy

```
sudo nano /etc/systemd/system/conuar.service
```

conuar.service – paste this content

Copy

```
[Unit]
Description=Conuar Inspection Webapp (Django)
After=network.target mysql.service

[Service]
```

```
User=YOUR_VM_USERNAME
WorkingDirectory=/opt/conuar/Conuar/conuar_webapp
ExecStart=/opt/conuar/Conuar/conuar_webapp/venv/bin/python manage.py runserver 0.0.0.0:8000
Restart=always
RestartSec=5
EnvironmentFile=/opt/conuar/Conuar/conuar_webapp/.env
StandardOutput=append:/opt/conuar/Conuar/conuar_webapp/logs/django.log
StandardError=append:/opt/conuar/Conuar/conuar_webapp/logs/django.log

[Install]
WantedBy=multi-user.target
```

Replace `YOUR_VM_USERNAME` with your actual VM username. Run `whoami` inside the VM to check it.

Enable and start the service

Copy

```
sudo systemctl daemon-reload
sudo systemctl enable conuar
sudo systemctl start conuar
sudo systemctl status conuar    # should show "active (running)"
```

ALTERNATIVE: CRON @REBOOT

If you prefer cron over systemd (note: no auto-restart on crash):

`crontab -e`

Copy

```
# Add this line inside crontab -e:
@reboot cd /opt/conuar/Conuar/conuar_webapp && \
/opt/conuar/Conuar/conuar_webapp/venv/bin/python manage.py runserver 0.0.0.0:8000 \
>> logs/django.log 2>&1 &
```

13

Set up the ETL monitor as a service

Keeps the PLC data processor running alongside Django

Create ETL service file

Copy


```
sudo nano /etc/systemd/system/conuar-etl.service
```

conuar-etl.service – paste this content

Copy

```
[Unit]
Description=Conuar PLC ETL Monitor
After=network.target mysql.service conuar.service

[Service]
User=YOUR_VM_USERNAME
WorkingDirectory=/opt/conuar/Conuar/conuar_webapp
ExecStart=/opt/conuar/Conuar/conuar_webapp/venv/bin/python etl/setup_plc_monitor.py
Restart=always
RestartSec=10
EnvironmentFile=/opt/conuar/Conuar/conuar_webapp/.env
StandardOutput=append:/opt/conuar/Conuar/conuar_webapp/logs/etl.log
StandardError=append:/opt/conuar/Conuar/conuar_webapp/logs/etl.log

[Install]
WantedBy=multi-user.target
```

Enable and start

Copy

```
sudo systemctl daemon-reload
sudo systemctl enable conuar-etl
sudo systemctl start conuar-etl
sudo systemctl status conuar-etl
```

14

Open the webapp in a browser

Access the application from any machine on the network

Browser URL

url

```
http://YOUR_VM_EXTERNAL_IP:8000
```

To find the external IP at any time:

Get external IP

Copy

```
# From inside the VM:
curl -s ifconfig.me

# From your local machine:
gcloud compute instances describe conuar-webapp \
  --zone=us-central1-a \
  --format='get(networkInterfaces[0].accessConfigs[0].natIP)'
```

Reserve a static IP so it does not change on VM restart: GCP Console → VPC Network → External IP addresses → Reserve Static Address.



Daily commands reference

Operations you will run regularly after initial deployment

Common operations

Copy

```
# View live Django logs (Ctrl+C to stop)
sudo journalctl -u conuar -f

# View live ETL logs
sudo journalctl -u conuar-etl -f

# Pull latest code and restart both services
cd /opt/conuar && git pull
sudo systemctl restart conuar
sudo systemctl restart conuar-etl

# Run Django migrations after a model change
cd /opt/conuar/Conuar/conuar_webapp
source venv/bin/activate
python manage.py migrate

# Check MySQL is running
sudo systemctl status mysql
```

```
# Connect to MySQL manually
mysql -u conuar -p conuar_webapp

# Check disk space
df -h
```



Summary

What was deployed and how each piece works

COMPONENT	CHOICE	NOTES
GCP compute	e2-medium	1 vCPU, 4 GB RAM, Debian 12, 30 GB boot disk
Network	tcp:8000	Firewall rule on http-server tag. Reserve static IP for production.
Database	MySQL 8.0	Installed on the VM. Dedicated conuar user, conuar_webapp DB.
Python env	venv	Isolated at /opt/conuar/Conuar/conuar_webapp/venv/
Django	runserver	Acceptable for internal tool. Managed by systemd conuar.service .
ETL monitor	systemd	PLC data processor runs as conuar-etl.service , restarts on crash.
Process management	systemd	Both services auto-start on boot and auto-restart on failure.
Secrets	.env	Loaded by python-dotenv . Never commit this file to git.
Browser access	http:// IP:8000	Direct access. Add Nginx + Let's Encrypt for HTTPS if needed later.